

Computer Network Defense

2026-10-07

Table of contents

0	Introduction	2
1	Patch Updates	2
1.1	The need for updates	2
1.2	The process of updating	2
2	Malware Protection	3
2.1	Types of malware	4
2.2	Protecting against malware	5
3	Firewalls	6
4	Other Defenses	7
4.1	Virtualization	7
4.2	Chroot jails	7
4.3	Docker Containers	8
5	Detection and Monitoring	8
5.1	Defense in depth	8
5.2	Logs	9
5.3	Intrusion detection and prevention systems	9
5.4	Centralized monitoring	10
5.5	Limiting the damage	10
6	Basic Offense	11

0 Introduction

As the name suggests, computer network defense is a set of actions that a body/group takes to make sure that there is no unauthorized activity within its computer network. It is basically the defensive side of Cyber warfare. A successful defense strategy prevents any attacks and exploits.

Computer network attacks (CNA) include lots of possibilities ranging from taking down systems, to modifying the data on a system, to misusing resources – basically anything that disrupts the availability and/or integrity of a system can be called an attack. **Computer network exploitation (CNE)**, on the other hand, targets the confidentiality of a system: the goal is to quietly gain access and gather information rather than to cause damage. If data is stolen or published, then that would fall under the exploitation category.

We shall start our discussion of defense with the three things EVERY computer user should definitely be doing to every system they use.

1 Patch Updates

1.1 The need for updates

Because systems are designed by humans, there are ALWAYS vulnerabilities being found and/or misused. As we discussed earlier in the quarter, a vulnerability can exist in the “wild” for months, and sometimes years, before being discovered. During this time, nefarious characters can use those vulnerabilities to carry out attacks on a system. After the vulnerability is discovered, it will take time to design and then publish a fix for that vulnerability. During this phase, information on the vulnerabilities and how they can be misused is typically exchanged by bad actors. After a patch is released, it is still up to the user to download and/or install that patch and from your personal experience, you know that that might take a long time. As users of these systems, we have very little control over the time it takes to identify and fix a vulnerability. However, we do have control over how long it will take up to download and install that update. Until the patch is installed, all users aware of the vulnerability can use it to attack your system.

Bottom line is, **install security updates as soon as possible.**

1.2 The process of updating

For a Linux system, the process of manually updating your system involves 4 commands. You should find a way of carrying out these commands as regularly as you can without it significantly affecting your system availability.

```
sudo apt update      # get the latest software inventory
sudo apt upgrade      # based on the inventory, make any updates to
                      # software I have on my system

sudo apt dist-upgrade # sometimes there might be some low level
                      # software that needs upgrading too.

sudo apt autoremove   # remove old software or installation files
                      # that are no longer being used.
```

Tip

The first two commands shown above are the most important. The upgrade command will take some time if it's been a while since your last upgrade.

Note

On Debian/Ubuntu systems, security updates can be installed automatically with the **unattended-upgrades** package. This is one way of making sure updates happen regularly without having to remember to run the commands above.

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

2 Malware Protection

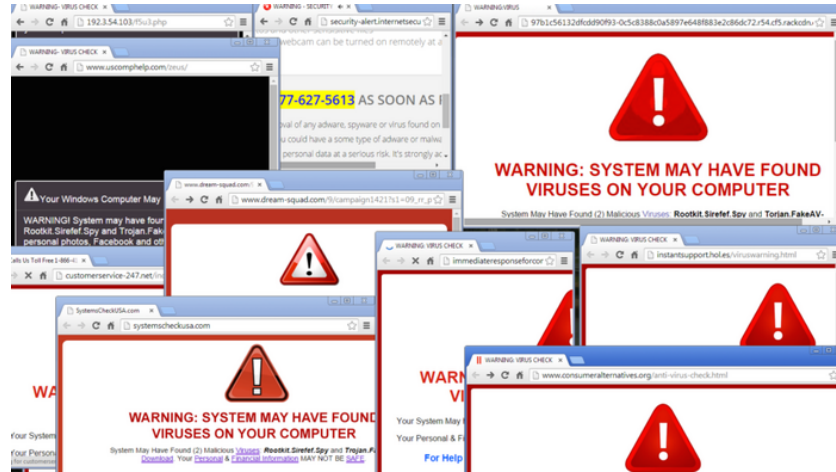
“Malicious Software”, or **malware**, is an all encompassing word for any software designed explicitly to exploit or attack a system. Some of the tell-tale signs that your system has malware include:

- slow or crashing system,
- little to no storage space,
- emails being sent without consent,
- pop-ups,
- programs opening, closing, or modifying data on their own,
- etc.

2.1 Types of malware

There are a few different types/kinds of malware in existence. However it is important to realize that while these types might have different definitions, there are multiple examples of malware that could be classified under multiple categories.

1. **Viruses** - Small piece of code inserted into seemingly innocuous application or system and are typically deployed by the victim themselves. Easily spreads to other files on that system and can do things such as destroy data, steal data, send infected files to contact lists, launch other attacks on other victims.
2. **Worms** - Very similar to viruses but do not need human interaction to make a copy of itself and doesn't need to attach itself to a specific program to cause damage. Worms can be transmitted via email attachments, removable media, etc.
3. **Bacteria** - This is another name for the fork bomb (or rabbit virus) similar to the one you saw earlier in the quarter. This is a program that when run takes over system resources by replicating itself. This typically results in slower systems, or even crashing the system which might seem like a small price for a personal computer but is a very high price for other systems that run equipment that should never be off.
4. **Trojans** - Malware that masquerades as a benign program in order to persuade a victim to install it. Unlike viruses and worms, trojans do not replicate themselves. Trojans are named after the famous Trojan horse of Greek mythology.
5. **Rootkits** - Software that helps cyber criminals or other malware to remain hidden. They will do things like grant access of the system to cyber criminals, hide evidence of such access, etc. They don't typically cause serious damage to the system directly but they make it easier for other software or people to do so.
6. **Spyware** - Spyware's purpose, as the name suggests, is to spy on the user of the system. They will do things like track internet or system activity for any sensitive information e.g. passwords.
7. **Adware** - Malware that supports advertising. Sometimes this will look like pop-up ads. Adware also typically is used to provide data on your system usage to its designer. Not all adware is malicious but they can definitely be annoying.
8. **Ransomware** - Malware that locks and/or encrypts a user's system and requires a payment (usually in bitcoin) to regain access to the system.



While there are lots of types of malware, they are all typically transmitted in a similar manner and that allows us to either avoid them or mitigate their effect. The 2019 Verizon Data Breach Investigations Report found that 94% of malware was delivered by email. Other ways include manual exploitation of security vulnerabilities, drive-by downloads, social engineering (phishing, vishing, smishing, etc.)

2.2 Protecting against malware

How does one protect themselves against malware? The only foolproof way is to never go on the internet and never use a removable device. But that is too high a price to pay. A more realistic price is being vigilant. At the beginning of class, we mentioned how the weakest link in any cyber security system is the human being and this is one of the reasons. CISA (Cybersecurity and Infrastructure Security Agency) is pushing for everyone to activate **multi-factor authentication** (MFA). Much malware, and especially phishing, exists to steal passwords. With MFA enabled, a stolen password alone is not enough to log into your accounts, which makes it much harder for an attacker to take over your email and use it to spread malware to your contacts. Phishing-resistant forms of MFA such as hardware security keys and passkeys are stronger than codes sent by text message, since those codes can themselves be phished.

MFA limits what an attacker can do with stolen credentials, but it doesn't stop a victim from getting malware in the first place. Being **skeptical and vigilant** can help one avoid suspicious links, websites, and emails. Another way is to tinker with your spam folder settings so that emails promising you free iPads will never show up in your inbox to begin with.

Patch updates. We have already talked about this but it is worth mentioning here again. The success or failure of a downloaded piece of malware will depend on whether you have the appropriate updates or not.

Installing and updating anti-malware software is another very important thing one can do to defend against malware. Do your research and find the software that fits within your budget and does what you want it to do.

A final possible fall back is **maintaining backups** of your data. If all else fails, at the very least you can format your system and restore the data you have backed up and only lose a few hours or a few days of data and not everything. This is a good habit to have for more than just avoiding malware.

3 Firewalls

A **firewall** is a security system that monitors and controls incoming and outgoing network traffic based on a set of rules. Firewalls act as a barrier between a trusted internal network and untrusted external networks. This allows us to stop active cyber criminals from accessing a system or a home network. Firewalls can also stop malware on your system or network from communicating and therefore propagating to other systems and/or their designer.

There are two common ways of writing firewall rules. The first is **application-based**: connections are allowed or blocked depending on the application that created them. This is the approach you will typically interact with on Windows, where you are asked whether a program should be allowed to communicate on the network. One downside of this is that you have to interact with the firewall a little more since you have to change the settings for any software you want to use that requires an internet connection.

The second is **port-based**: connections are allowed or blocked based on the port (and protocol, address, etc.) that they are using. This is the approach typically used on Linux, where tools such as UFW are front ends to the kernel's packet filtering framework (netfilter, configured with **nftables** or **iptables**). In practice, both operating systems can do both: the Windows firewall supports port-based rules, and Linux can be configured to filter by application. As you might know from previous classes, port numbers are 16 bits long, so there are 65,536 (2^{16}) ports, numbered 0–65535, for each of TCP and UDP. Ports 0–1023 are the **well-known** (system) ports and are typically used for common services e.g. HTTP on port 80, HTTPS on 443, SSH on 22, DNS on 53, FTP on 20 and 21, IMAP on 143, etc. Ports 1024–49151 are **registered** ports used by specific applications, and ports 49152–65535 are **dynamic** (ephemeral) ports that operating systems hand out temporarily, e.g. for the client side of a connection. Later on in this class, you will set up a Linux firewall in an ingenious way to lock out all connections that do not know about a “secret knock” of yours.

4 Other Defenses

While the three techniques we discussed above should be required for all systems, they are not 100% infallible especially since we are dealing with humans. There are times when it would be helpful and beneficial to implement other defensive strategies.

4.1 Virtualization

One of the techniques you have already used in this class is virtualization. Virtualization allows us to isolate a portion of the computer and install a completely separate system on it. This is the approach most people will use when they have to deal with inherently risky files (e.g. studying viruses), or connect to risky networks. This allows us to have explicit control over what kinds of files might be corrupted or affected by nefarious actors. It also allows us to easily revert to an older snapshot of your computer if something goes awry.

The only con is you need to be technically savvy and spend a lot of time to set up and maintain a virtual machine, especially if the default settings don't work for you. Also, because it is only allowed to access part of your original system e.g. less RAM, it can be slow and cumbersome to work with.

4.2 Chroot jails

As background, when you open a file manager on a Linux machine, you are typically put in the home folder. Most of a person's usual work will be based at home. However, if you go two levels up you get to the root folder at the filepath `/`, and this is the grandparent of all files on the system.

In the root folder at `/` You can find files used to boot the computer at `/boot/`, programs installed on the computer at `/bin/`, all devices connected to the computer at `/dev/`, and more. Understandably this is a lot of information that anyone who logs into your computer might have access to.

A way to deal with the vulnerability of being able to access the root folder is to create a **chroot jail**. The name chroot comes from "change root" which essentially means that you change the root folder for some users. This allows you to create a small subsystem of folders that that user has access to and can never get out of. That is, a **chroot jail** is a way of isolating or restricting access for users of a system. Setting up a chroot jail even means that you determine which programs and utilities a user will be able to access in their jail e.g. can they use `ls`, `mkdir`, etc.

Warning

A chroot jail only changes what a process sees as /. It is not a complete security boundary on its own: a process running as **root** inside the jail can escape it. Chroot jails should therefore only be used to confine unprivileged users and processes.

We discuss and implement chroot jails later in this class. For those of you who have been trying out bandit, this is probably one of the ways that system is set up since you cannot see any folders other than what the designer of the system gives you access to.

Setting chroot jails up takes way less system resources than virtualization and is quicker but the jail will have the same OS as the host and a subset of the tools installed on the host.

4.3 Docker Containers

Docker containers are a more recent entry into the isolation game. They allow users to set up environments with operating systems, and software that are portable to other users and systems and serve to give users an isolated environment that has been set up and tested by someone else. It reduces the amount of work that the end user will have to do to get their computer set up to tackle a task.

Isolation software or approaches aren't the only way to defend against attacks online. Unfortunately, there are way too many approaches to discuss in this class but I hope that this has introduced you to a few of the main ones that anyone can use on their own systems and perhaps even some enterprise systems.

5 Detection and Monitoring

Everything we have discussed so far has been about *prevention*: keeping attackers and malware out in the first place. Unfortunately, prevention will eventually fail. A patch might not exist yet, a user might click on a malicious link, or a firewall rule might be written incorrectly. A good defender therefore assumes that something will get through and makes sure that they will **notice** when it does. The longer an attacker goes unnoticed on a network, the more time they have to steal data, spread to other systems, and cover their tracks.

5.1 Defense in depth

Defense in depth is the idea of layering several independent defenses so that the failure of any single one does not lead to a breach. Think of a castle: it has a moat, outer walls, inner walls, guards, and a locked treasury. An attacker who crosses the moat still has to get through everything else. The techniques in these notes are layers of this kind: patches, anti-malware

software, firewalls, isolation, and finally monitoring, which is how we find out that an attacker has made it through the other layers.

5.2 Logs

Most activity on a system leaves a record somewhere. On Linux, system services write their messages to the systemd journal, which is read with the `journalctl` command. Many distributions also keep plain-text logs in `/var/log/`. For example, `/var/log/auth.log` on Ubuntu records logins, uses of `sudo`, and SSH connection attempts.

```
sudo journalctl -u ssh                # messages from the SSH server
sudo grep "Failed password" /var/log/auth.log # failed SSH logins
```

Logs are only useful if someone (or something) actually reads them. Attackers know this too: one of the first things an attacker with root access will often do is delete or edit the logs to hide what they did. This is one reason organizations copy their logs to a separate machine as soon as they are written.

5.3 Intrusion detection and prevention systems

An **intrusion detection system (IDS)** monitors activity and raises an alert when it sees something suspicious. An **intrusion prevention system (IPS)** goes a step further: it sits in the path of the traffic and blocks whatever it believes is malicious. The trade-off is that an IDS that is wrong only produces a bad alert, while an IPS that is wrong can block legitimate users.

There are two main approaches to deciding what is suspicious.

1. **Signature-based** detection compares activity against a database of patterns from known attacks, much like anti-malware software does. This is accurate for attacks that have been seen before but cannot catch attacks that are new.
2. **Anomaly-based** detection learns what “normal” looks like for a system or network and flags anything that deviates from it, e.g. a workstation that suddenly sends gigabytes of data overseas at 3 a.m. This can catch new attacks but tends to raise more false alarms.

Detection systems are also described by where they are placed. A **network-based IDS (NIDS)**, such as Snort or Suricata, watches the traffic passing through a point in the network, the same kind of traffic you will see with Wireshark. A **host-based IDS (HIDS)**, such as Wazuh or AIDE, runs on an individual machine and watches its logs, processes, and files for changes. Note that a NIDS cannot see inside encrypted traffic such as HTTPS, which is one reason host-based monitoring is still needed.

i Note

Every detection system makes two kinds of mistakes. A **false positive** is an alert for something that was actually harmless, and a **false negative** is an attack that was missed. Tuning a system to catch more attacks usually means more false positives, and analysts who are buried in false alarms start ignoring alerts altogether (*alert fatigue*).

A simple example of automated detection and response on a single machine is **fail2ban**. It watches log files for repeated failed logins and, when one address fails too many times, adds a firewall rule that bans that address for a period of time. You will set this up later in this class.

5.4 Centralized monitoring

In an organization with hundreds or thousands of machines, nobody can read the logs on each one. A **Security Information and Event Management (SIEM)** system collects logs and alerts from across the organization into one place, where they can be searched and connected. A single failed login is not interesting, but the same address failing to log into fifty different servers in five minutes is. Examples include Splunk, the Elastic Stack, and Wazuh. In larger organizations, a team of analysts in a **Security Operations Center (SOC)** watches these systems and responds to alerts.

5.5 Limiting the damage

Since we assume that some attacks will succeed, it also makes sense to limit how much damage a successful attacker can do.

- **Least privilege** means giving every user and program only the access it needs to do its job and nothing more. Logging in as a regular user instead of root, and only adding trusted administrators to powerful groups such as **docker** or **sudo**, are examples.
- **Network segmentation** means splitting a network into separate zones with firewalls between them, so that an attacker who compromises one zone cannot directly reach the others. For example, a company's guest Wi-Fi should not be able to reach its payroll servers. Taken to its extreme, this leads to **zero trust** networks, where no device is trusted just because it is on the internal network, and every request must be authenticated.

6 Basic Offense

Sometimes a good defense is based on understanding the capabilities of the offense. Unfortunately, the offense is always growing and wrapping your mind round all the possibilities is a daunting task. Nonetheless, let us discuss a couple of things that are commonly done during an offense i.e. recon/footprinting tactics.

To look for vulnerabilities (a process done by both attackers and defenders of a network) some tools can be used. **Nmap** is probably the most common but others include **hping3**, **scapy** (a Python library, installed with the **python3-scapy** package), **netcat**, **telnet**. For many of these tools, there is some overlap in what they do but the more you work with them, the more you'll get a feeling for which tools work best in what situation. Familiarity with the tool and its capabilities is one of those factors that will also affect your choice of tool.

```
sudo apt install nmap
nmap <address or address range>
nmap -h                # to see some of the options
nmap -Pn <address>      # skip host discovery (treat host as up)
sudo nmap -O <address>  # OS detection (requires root)
```

! Important

Only scan networks and machines that you own or have **explicit written permission** to test. Scanning systems without authorization can violate laws such as the Computer Fraud and Abuse Act as well as university and ISP policies.

Another offensive operation is network sniffing. This involves just looking at network traffic and then figuring out more specific targets based on that. The most common tool is **wireshark**.

This is supposed to be small preview of a few tools. We shall dive deeper into some of them in later lectures. But now that you have heard about them, nothing stops you from taking a look at a few of them on your own time.

References:

1. [Computer Network Exploitation \(ScienceDirect Topics\)](#)
2. [Computer Network Attack \(ScienceDirect Topics\)](#)
3. [Computer Network Defense \(ScienceDirect Topics\)](#)
4. [Types of Malware \(Norton\)](#)